



LABORATORIO DI RETI INFORMATICHE

CORSO DI LAUREA TRIENNALE IN INGEGNERIA INFORMATICA
A.A. 2025/2026

Francesca Righetti
francesca.righetti@unipi.it



ESERCITAZIONE 3

Configurazione di DHCP e test di connettività

Programma

- Configurazione di DHCP
 - Lato server e lato client
- Test di connettività
- Test del DNS
- Analisi dei pacchetti

DYNAMIC HOST CONFIGURATION PROTOCOL (DHCP)

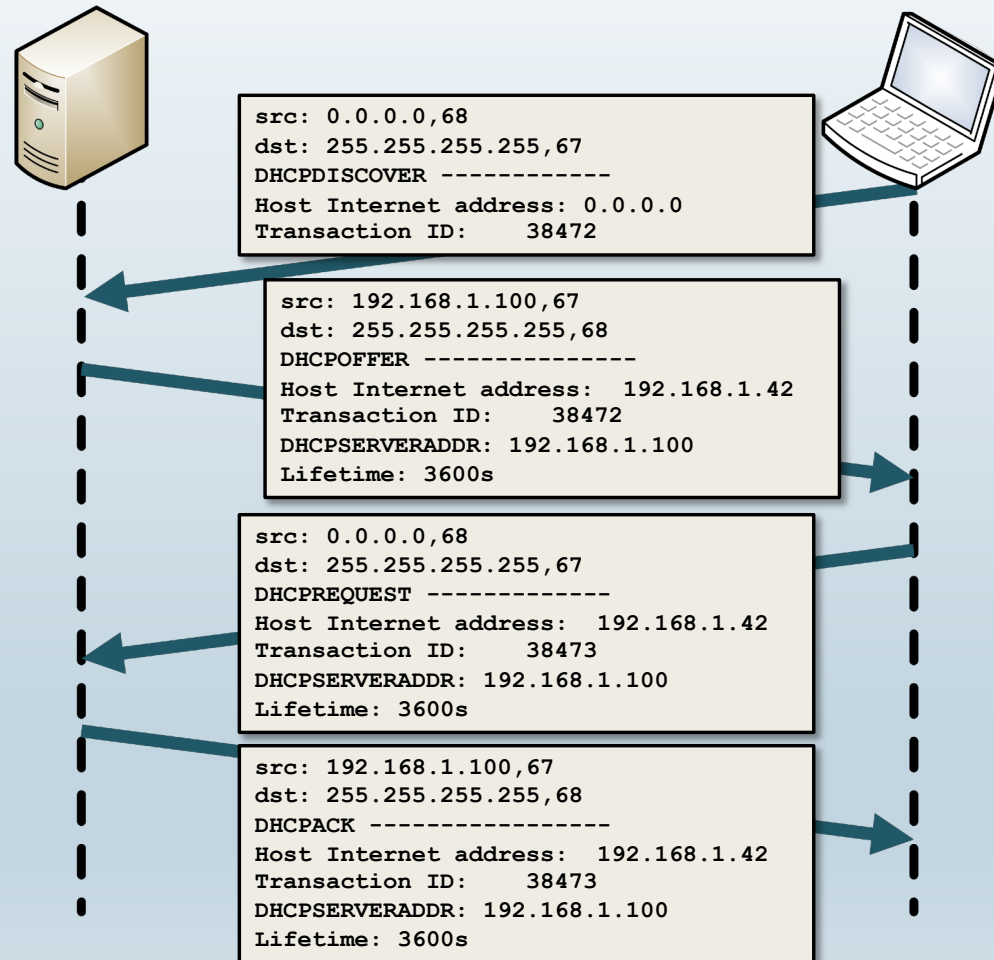
DHCP

- La configurazione manuale richiede molto tempo e si presta a errori
- Il Dynamic Host Configuration Protocol (DHCP) consente la configurazione automatica e dinamica dei parametri TCP/IP degli host, UDP porta 67:
 - Indirizzo IP
 - Maschera di rete
 - Indirizzo del gateway
 - Indirizzo del server DNS
- All'interno della rete è configurato un server DHCP
- Quando il client si connette alla rete, il server gli fornisce i parametri di configurazione
- L'indirizzo IP viene scelto da un pool di indirizzi disponibili
- Le informazioni hanno una scadenza, così gli indirizzi IP bloccati da client che si sono disconnessi possono essere riutilizzati

DHCP Server DHCP (1+)

Processo di leasing IP

- **DORA**
 - Discover
 - Offer
 - Request
 - Acknowledge



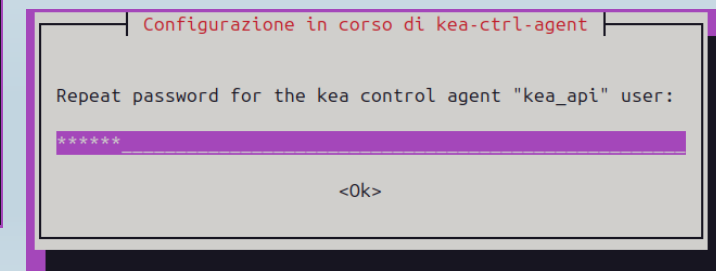
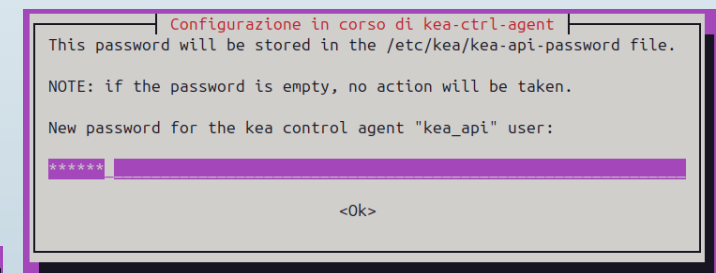
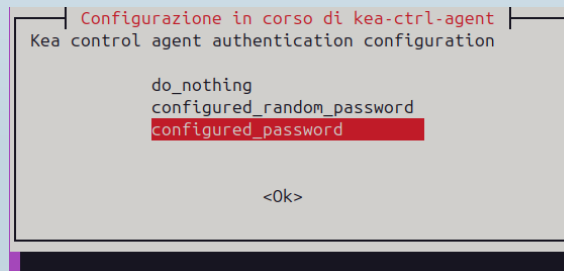
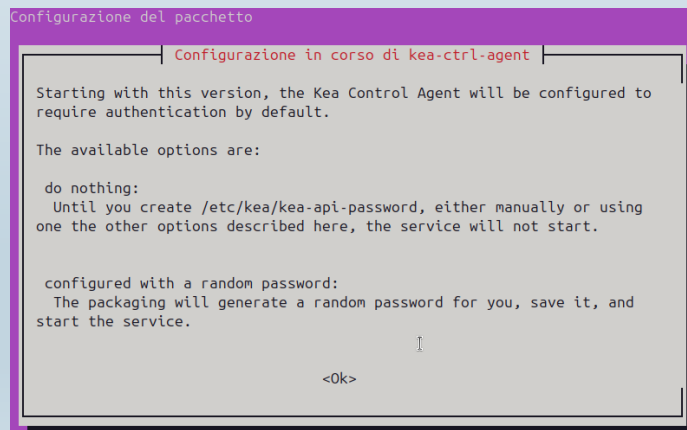
Server DHCP - Kea

■ Kea DHCP

- Software open source per DHCP server, che sostituisce ISC DHCP (end of life in 2022)
- È più orientato ad ambienti di rete moderni
- Dove si usa ?
 - ISP, grandi aziende, ambienti cloud, datacenter, laboratori

■ Installazione

- `sudo apt install kea`



Server DHCP - Kea

```
// This is just a very basic configuration. Kea comes with large suite (over 30)
// of configuration examples and extensive Kea User's Guide. Please refer to
// those materials to get better understanding of what this software is able to
// do. Comments in this configuration file sometimes refer to sections for more
// details. These are section numbers in Kea User's Guide. The version matching
// your software should come with your Kea package, but it is also available
// in ISC's Knowledgebase (https://kea.readthedocs.io; the direct link for
// the stable version is https://kea.readthedocs.io/).
//
// This configuration file contains only DHCPv4 server's configuration.
// If configurations for other Kea services are also included in this file they
// are ignored by the DHCPv4 server.
//
// DHCPv4 configuration starts here. This section will be read by DHCPv4 server
// and will be ignored by other components.
Dhcp4: {
    // Add names of your network interfaces to listen on.
    "interfaces-config": {
        // See section 8.2.4 for more details. You probably want to add just
        // interface name (e.g. "eth0" or specific IPv4 address on that
        // interface name (e.g. "eth0/192.0.2.1").
        "interfaces": [ ]

        // Kea DHCPv4 server by default listens using raw sockets. This ensures
        // all packets, including those sent by directly connected clients
        // that don't have IPv4 address yet, are received. However, if your
        // traffic is always relayed, it is often better to use regular
        // UDP sockets. If you want to do that, uncomment this line:
        // "dhcp-socket-type": "udp"
    },

    // Kea supports control channel, which is a way to receive management
    // commands while the server is running. This is a Unix domain socket that
    // receives commands formatted in JSON, e.g. config-set (which sets new
    // configuration), config-reload (which tells Kea to reload its
    // configuration from file), statistic-get (to retrieve statistics) and many
    // more. For detailed description, see Sections 8.8, 16 and 15.
    "control-socket": {
        "socket-type": "unix",
        "socket-name": "/run/kea/kea4-ctrl-socket"
    },

    // Use Memfile lease database backend to store leases in a CSV file.
    // Depending on how Kea was compiled, it may also support SQL databases
    // (MySQL and/or PostgreSQL). Those database backends require more
    // parameters, like name, host and possibly user and password.
    // There are dedicated examples for each backend. See Section 7.2.2 "Lease
    // Storage" for details.
    "lease-database": {
        // Memfile is the simplest and easiest backend to use. It's an in-memory
        // C++ database that stores its state in CSV file.
        "type": "memfile",
        "lfc-interval": 3600
    }
}
```

■ Kea DHCP

- File di configurazione: /etc/kea/kea-dhcp4.conf

■ Se si vuole configurare un server DHCP, dopo le modifiche al file di configurazione, va riavviato il servizio

- systemctl restart kea-dhcp4-server.service

■ Per consultare lo stato del server, e controllare eventuali errori

- systemctl status kea-dhcp4-server.service
- sudo journalctl -u kea-dhcp4-server -b

Server DHCP - Kea

```
"Dhcp4": {  
  // Add names of your network interfaces to listen on.  
  "interfaces-config": {  
    // See section 8.2.4 for more details. You probably want to add just  
    // interface name (e.g. "eth0" or specific IPv4 address on that  
    // interface name (e.g. "eth0/192.0.2.1").  
    "interfaces": [ ]  
  
    // Kea DHCPv4 server by default listens using raw sockets. This ensures  
    // all packets, including those sent by directly connected clients  
    // that don't have IPv4 address yet, are received. However, if your  
    // traffic is always relayed, it is often better to use regular  
    // UDP sockets. If you want to do that, uncomment this line:  
    // "dhcp-socket-type": "udp"  
  },  
  
  // Kea supports control channel, which is a way to receive management  
  // commands while the server is running. This is a Unix domain socket that  
  // receives commands formatted in JSON, e.g. config-set (which sets new  
  // configuration), config-reload (which tells Kea to reload its  
  // configuration from file), statistic-get (to retrieve statistics) and many  
  // more. For detailed description, see Sections 8.8, 16 and 15.  
  "control-socket": {  
    "socket-type": "unix",  
    "socket-name": "/run/kea/kea4-ctrl-socket"  
  },  
}
```

- “interfaces-config”
 - Sezione che configura su quali interfacce di rete il server DHCP ascolta.
- “interfaces”
 - Elenco delle interfacce in cui il server DHCP riceve e invia pacchetti DHCP.
 - ["eth0"] per ascoltare su eth0
 - ["eth0/192.168.10.1"] per vincolare all'IP 192.168.10.1 su eth0.
 - ["*"] per tutte le interfacce, sconsigliato se la macchina ha molte interfacce di rete.
- ‘control-socket’
 - Abilita il canale di controllo locale, per inviare comandi a runtime (config-reload, statistic-get, ecc.) senza riavviare il demone.

Server DHCP - Kea

```
"lease-database": {
    // Memfile is the simplest and easiest backend to use. It's an in-memory
    // C++ database that stores its state in CSV file.
    "type": "memfile",
    "lfc-interval": 3600
},

// Kea allows storing host reservations in a database. If your network is
// small or you have few reservations, it's probably easier to keep them
// in the configuration file. If your network is large, it's usually better
// to use database for it. To enable it, uncomment the following:
// "hosts-database": {
//     "type": "mysql",
//     "name": "kea",
//     "user": "kea",
//     "password": "kea",
//     "host": "localhost",
//     "port": 3306
// },
// See Section 7.2.3 "Hosts storage" for details.

// Setup reclamation of the expired leases and leases affinity.
// Expired leases will be reclaimed every 10 seconds. Every 25
// seconds reclaimed leases, which have expired more than 3600
// seconds ago, will be removed. The limits for leases reclamation
// are 100 leases or 250 ms for a single cycle. A warning message
// will be logged if there are still expired leases in the
// database after 5 consecutive reclamation cycles.
// If both "flush-reclaimed-timer-wait-time" and "hold-reclaimed-time" are
// not 0, when the client sends a release message the lease is expired
// instead of being deleted from the lease storage.
"expired-leases-processing": {
    "reclaim-timer-wait-time": 10,
    "flush-reclaimed-timer-wait-time": 25,
    "hold-reclaimed-time": 3600,
    "max-reclaim-leases": 100,
    "max-reclaim-time": 250,
    "unwarned-reclaim-cycles": 5
},
```

■ "lease-database"

- Dove e come il server DHCP salva gli assegnamenti di indirizzi IP
- "memfile" imposta il salvataggio su file
 - /var/lib/kea
- "lfc-interval"
 - Intervallo per la compattazione periodica dei file di lease

■ "expired-leases-processing"

- reclaim-timer-wait-time: Ogni 10 s ciclo che individua lease scaduti.
- flush-reclaimed-timer-wait-time: Ogni 25 s rimuove in blocco dal file i lease scaduti da più di hold-reclaimed-time.
- hold-reclaimed-time: Tiene i lease scaduti per 1 ora prima di eliminarli.
- max-reclaim-leases: Per ciclo processa al massimo 100 lease, evita picchi CPU.
- max-reclaim-time: 250
- unwarned-reclaim-cycles: Dopo 5 cicli in cui restano ancora tanti lease scaduti, logga un warning.

Server DHCP - Kea

```
// Global timers specified here apply to all subnets, unless there are
// subnet specific values defined in particular subnets.
"renew-timer": 900,
"rebind-timer": 1800,
"valid-lifetime": 3600,
"option-data": [
  // When specifying options, you typically need to specify
  // one of (name or code) and data. The full option specification
  // covers name, code, space, csv-format and data.
  // space defaults to "dhcp4" which is usually correct, unless you
  // use encapsulate options. csv-format defaults to "true", so
  // this is also correct, unless you want to specify the whole
  // option value as long hex string. For example, to specify
  // domain-name-servers you could do this:
  // {
  //   "name": "domain-name-servers",
  //   "code": 6,
  //   "csv-format": "true",
  //   "space": "dhcp4",
  //   "data": "192.0.2.1, 192.0.2.2"
  // }
  // but it's a lot of writing, so it's easier to do this instead:
  {
    "name": "domain-name-servers",
    "data": "192.0.2.1, 192.0.2.2"
  },

  // Typically people prefer to refer to options by their names, so they
  // don't need to remember the code names. However, some people like
  // to use numerical values. For example, option "domain-name" uses
  // option code 15, so you can reference to it either by
  // "name": "domain-name" or "code": 15.
  {
    "code": 15,
    "data": "example.org"
  },
],
```

- Timer che vengono notificati dal server al client:
 - “renew-timer”
 - Dopo 900 s il client prova a rinnovare con il server che gli ha dato il lease.
 - “rebind-timer”
 - Dopo 1800 s, se il rinnovo non riesce, il client ritenta in broadcast verso qualunque server
 - “valid-lifetime”
 - Durata del lease in secondi. Qui 1 ora.
- “option-data”
 - Opzioni globali. La gerarchia è: globale < classe < subnet < host. Un livello più specifico sovrascrive quello più generico.
 - Nel file ci sono altre opzioni globali che qui omettiamo.

Server DHCP - Kea

```
"client-classes": [
{
  // This specifies a name of this class. It's useful if you need to
  // reference this class.
  "name": "voip",

  // This is a test. It is an expression that is being evaluated on
  // each incoming packet. It is supposed to evaluate to either
  // true or false. If it's true, the packet is added to specified
  // class. See Section 12 for a list of available expressions. There
  // are several dozens. Section 8.2.14 for more details for DHCPv4
  // classification and Section 9.2.19 for DHCPv6.
  "test": "substring(option[60].hex,0,6) == 'Aastra'",

  // If a client belongs to this class, you can define extra behavior.
  // For example, certain fields in DHCPv4 packet will be set to
  // certain values.
  "next-server": "192.0.2.254",
  "server-hostname": "hal9000",
  "boot-file-name": "/dev/null"

  // You can also define option values here if you want devices from
  // this class to receive special options.
}
],
```

- Classi per applicare opzioni o comportamenti a gruppi di client che soddisfano una condizione
 - Test: Espressione valutata su ogni pacchetto.

Server DHCP - Kea

```
"subnet4": [
{
    // This defines the whole subnet. Kea will use this information to
    // determine where the clients are connected. This is the whole
    // subnet in your network.

    // Subnet identifier should be unique for each subnet.
    "id": 1,

    // This is mandatory parameter for each subnet.
    "subnet": "192.0.2.0/24",

    // Pools define the actual part of your subnet that is governed
    // by Kea. Technically this is optional parameter, but it's
    // almost always needed for DHCP to do its job. If you omit it,
    // clients won't be able to get addresses, unless there are
    // host reservations defined for them.
    "pools": [ { "pool": "192.0.2.1 - 192.0.2.200" } ],

    // These are options that are subnet specific. In most cases,
    // you need to define at least routers option, as without this
    // option your clients will not be able to reach their default
    // gateway and will not have Internet connectivity.
    "option-data": [
        {
            // For each IPv4 subnet you most likely need to specify at
            // least one router.
            "name": "routers",
            "data": "192.0.2.1"
        }
    ]
},
],
```

- Le assegnazioni di indirizzi sono per subnet
- Opzioni specifiche della subnet
 - routers è il default gateway. Se non impostato, i client avranno IP ma non sapranno dove instradare il traffico esterno.

```
"reservations": [
    // This is a reservation for a specific hardware/MAC address.
    // It's a rather simple reservation: just an address and nothing
    // else.
    {
        "hw-address": "1a:1b:1c:1d:1e:1f",
        "ip-address": "192.0.2.201"
    },

    // This is a reservation for a specific client-id. It also shows
    // the this client will get a reserved hostname. A hostname can
    // be defined for any identifier type, not just client-id.
    {
        "client-id": "01:11:22:33:44:55:66",
        "ip-address": "192.0.2.202",
        "hostname": "special-snowflake"
    },

    // The third reservation is based on DUID. This reservation defines
    // a special option values for this particular client. If the
    // domain-name-servers option would have been defined on a global,
    // subnet or class level, the host specific values take preference.
    {
        "duid": "01:02:03:04:05",
        "ip-address": "192.0.2.203",
        "option-data": [ {
            "name": "domain-name-servers",
            "data": "10.1.1.202, 10.1.1.203"
        } ]
    }
],
```

Client DHCP

- La configurazione dell'indirizzo di un'interfaccia avviene con dhcp se nei file di configurazione è impostato a 'true'

```
network:  
  ethernets:  
    enp0s1:  
      dhcp4: true  
version: 2
```

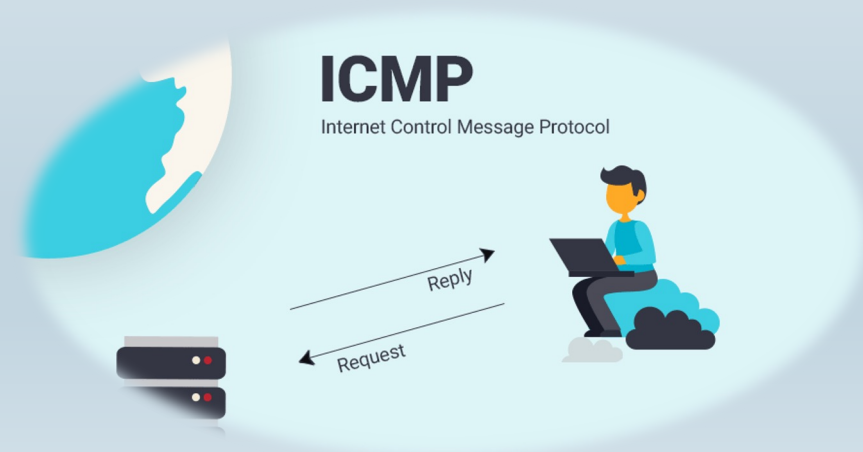


TEST DI CONNETTIVITÀ

PING e Traceroute

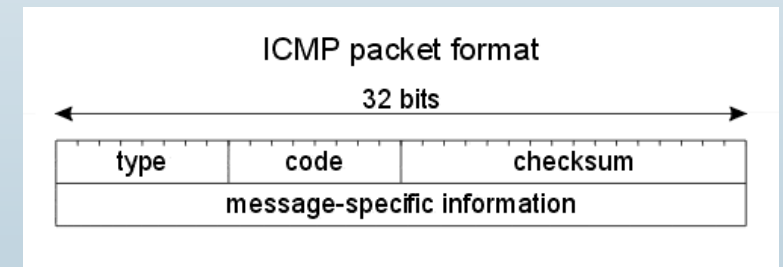
ICMP

- Internet Control Message Protocol (ICMP)
 - Protocollo della suite IP definito per lo scambio di messaggi di controllo, errore e diagnostica
 - Non trasporta dati applicativi
- ICMP serve per il rilevamento dei malfunzionamenti, ma non effettua nessuna correzione degli errori



ICMP

- Un pacchetto ICMP è incapsulato in un pacchetto IP e contiene:
 - Type (8 bit): identifica la categoria del messaggio
 - 0: Echo reply
 - 3: Destination network (code: 0)/host (code:1) unreachable
 - 8 Echo request
 - 11: Time to Live expired
 - 30 Traceroute
 - ...
 - Code (8 bit): dettaglio specifico del tipo.
 - Checksum: controllo di integrità.
 - Data: variabili secondo il tipo.
- Esempio:
 - Echo Request: Type = 8, Code = 0.
 - Echo Reply: Type = 0, Code = 0.



ICMP

- Messaggi informativi

- Usati per diagnosi e controllo:

- Echo Request / Echo Reply (Type 8 / 0) → usati da ping.
 - Router Advertisement / Router Solicitation (in ICMPv6 più che in ICMPv4), per l'autoconfigurazione di rete.

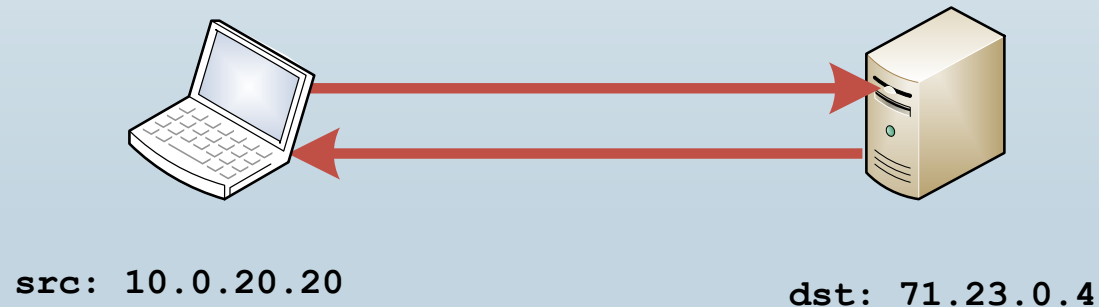
- Messaggi di errore

- Usati per segnalare problemi di consegna o instradamento:

- Destination Unreachable (Type 3)
 - Code 0: network
 - Code 1: host
 - Code 3: port
 - TTL Exceeded (Type 11) Usato quando il campo TTL di un pacchetto scade. È fondamentale per il comando traceroute.

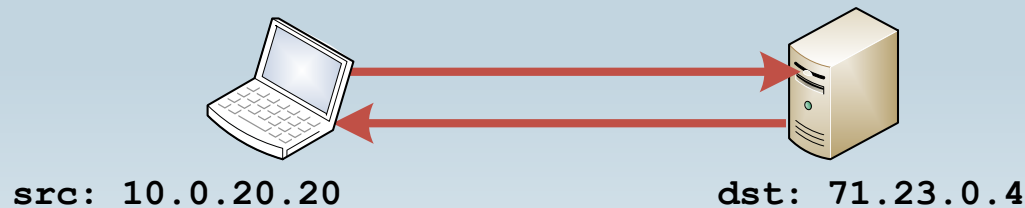
ICMP – Comando ping

- Serve per testare la connettività tra l'host e un altro host remoto (man ping)
 - ping www.apple.com
 - Ping 192.168.3.34
 - Il comando invia uno o più messaggi ICMP di tipo Echo Request e attende messaggi di tipo Echo Reply



ICMP – Comando ping

- Il comando invia uno o più messaggi ICMP di tipo Echo Request e attende messaggi di tipo Echo Reply
 - **Invio:** Il comando ping crea un pacchetto ICMP Echo Request, che include:
 - Intestazione ICMP (Type = 8, Code = 0).
 - Identificatore e numero di sequenza
 - Dati
 - **Trasmissione:** Il pacchetto è incapsulato in un pacchetto **IP** e inviato al destinatario.
 - **Ricezione:** L'host di destinazione, se attivo e configurato per rispondere, genera un pacchetto **ICMP Echo Reply** (Type = 0, Code = 0).
 - **Risposta:** Il pacchetto ritorna all'host sorgente.
 - **Misurazione:** Il tempo tra l'invio e la ricezione è misurato e mostrato come RTT. Ping normalmente invia più richieste, calcolando valori **min/avg/max/mdev**.

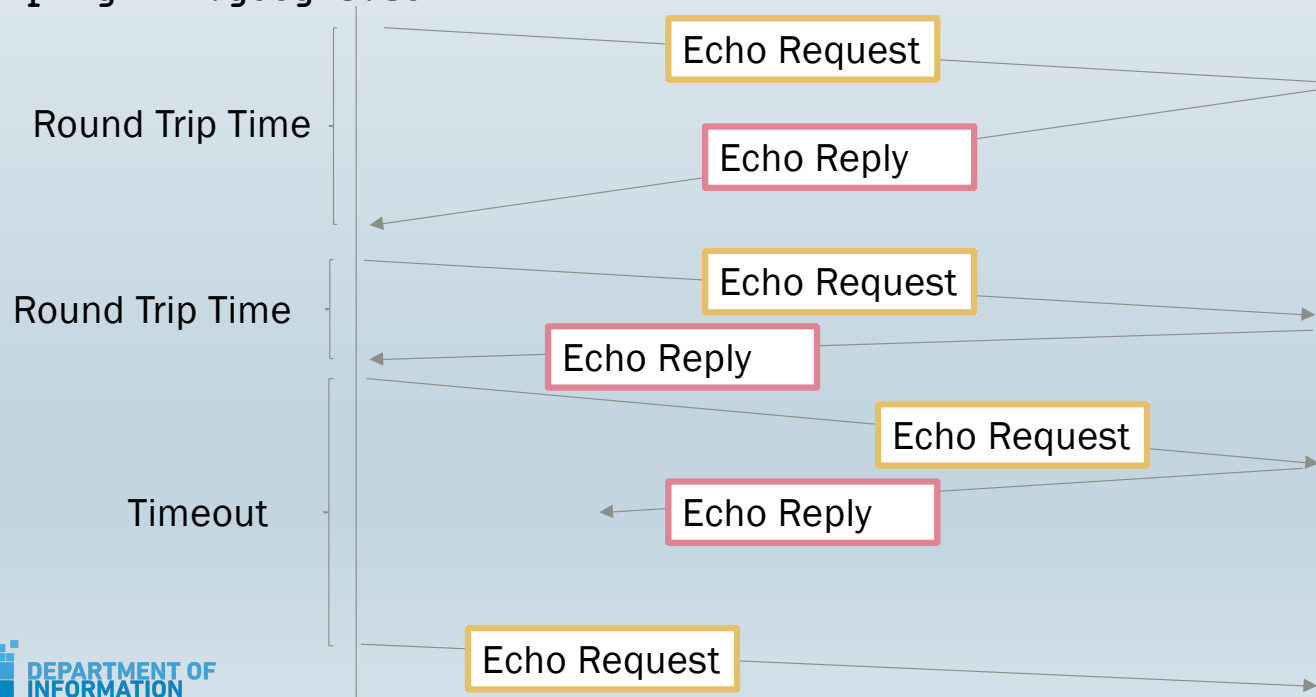


ICMP – Comando ping

www.google.com



ping www.google.com



ICMP – Comando ping

■ In dettaglio:

- ping su A invia a B una serie (di default, uno al secondo) di pacchetti Echo Request
- Quando B riceve un Echo Request, invia un pacchetto Echo Reply a A
- ping effettua il calcolo della percentuale dei pacchetti ricevuti e del Round Trip Time (RTT)
- Al termine del comando, presenta le statistiche
 - Se non specificato diversamente, il comando termina solo se l'utente lo interrompe con Ctrl+C

```
PING www.google.com (216.58.210.196) 56(84) bytes of data.  
64 bytes from mrs04s09-in-f4.1e100.net (216.58.210.196): icmp_seq=1 ttl=53 time=42.8 ms  
64 bytes from mrs04s09-in-f4.1e100.net (216.58.210.196): icmp_seq=2 ttl=53 time=32.2 ms  
64 bytes from mrs04s09-in-f4.1e100.net (216.58.210.196): icmp_seq=3 ttl=53 time=32.7 ms  
64 bytes from mrs04s09-in-f4.1e100.net (216.58.210.196): icmp_seq=4 ttl=53 time=35.8 ms  
64 bytes from mrs04s09-in-f4.1e100.net (216.58.210.196): icmp_seq=5 ttl=53 time=33.0 ms  
64 bytes from mrs04s09-in-f4.1e100.net (216.58.210.196): icmp_seq=6 ttl=53 time=32.6 ms  
64 bytes from mrs04s09-in-f4.1e100.net (216.58.210.196): icmp_seq=7 ttl=53 time=32.3 ms  
64 bytes from mrs04s09-in-f4.1e100.net (216.58.210.196): icmp_seq=8 ttl=53 time=33.2 ms  
64 bytes from mrs04s09-in-f4.1e100.net (216.58.210.196): icmp_seq=9 ttl=53 time=33.1 ms  
64 bytes from mrs04s09-in-f4.1e100.net (216.58.210.196): icmp_seq=10 ttl=53 time=32.1 ms  
  
--- www.google.com ping statistics ---  
10 packets transmitted, 10 received, 0% packet loss, time 9017ms  
rtt min/avg/max/mdev = 32.105/34.043/42.883/3.124 ms
```

ICMP – Comando ping

■ Possibili errori

- Network unreachable
 - L'host locale non ha route valide per raggiungere l'host remoto
- 100% packet loss
 - L'host locale non ha ricevuto nessun pacchetto di risposta
- Unknown host
 - Non è stato possibile risolvere il nome di host specificato

ICMP – Comando ping

■ Opzioni

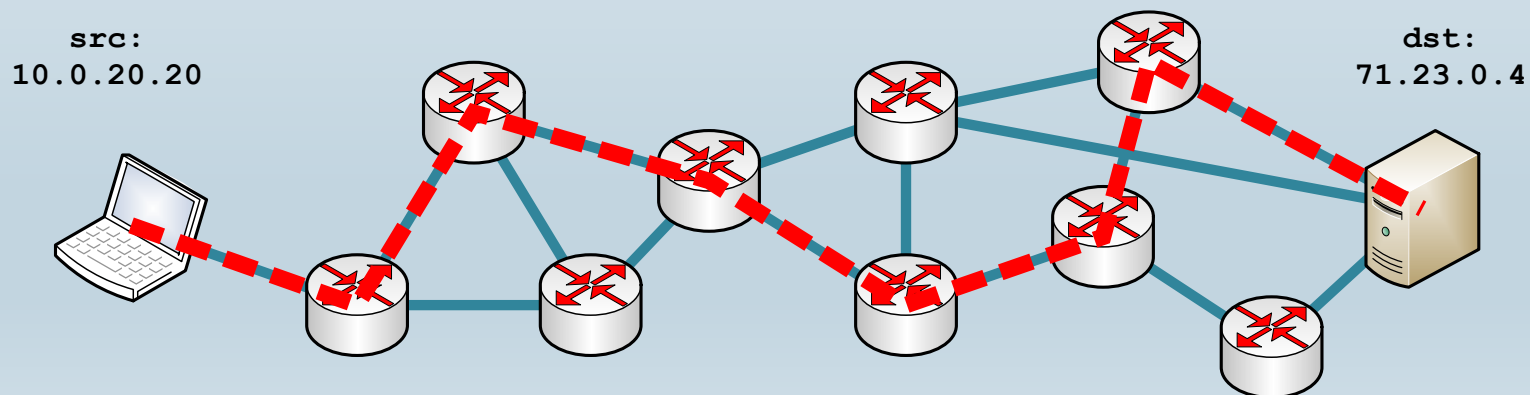
- -c (count)
 - Specifica il numero di richieste da inviare
- -i (interval)
 - Specifica l'intervallo tra le richieste
- -s (packetsize)
 - Specifica la dimensione del payload del pacchetto ICMP
- -f (flood ping)
 - Invia pacchetti ICMP il più velocemente possibile (una sorta di “stress test”). Usare con cautela, in quanto può sovraccaricare la rete o il server di destinazione. Richiede privilegi di amministratore.
- -W (timeout)
 - tempo massimo (in secondi) che ping aspetterà per una risposta prima di considerare il pacchetto perso.
- -t (Time To Live)
 - Specifica il valore del Time To Live del pacchetto. Questo parametro può essere utilizzato per tracciare il numero di hop che il pacchetto attraversa, identificando problemi legati al routing.
- -q (quiet)
 - Visualizza solamente le statistiche finali

ICMP – Comando ping - LIMITI

- Può essere bloccato da firewall per ridurre attacchi (es. ping flood, Denial of Service (DoS))
 - bloccare ICMP può far venir meno altre funzioni vitali di gestione della rete
- Misura solo la raggiungibilità a livello IP, non la disponibilità del servizio applicativo.
- Non distingue tra congestione di rete, filtri o host realmente spento.
- Cosa succede se eseguiamo ping così?
 - ping unipi.it
 - ping www.unipi.it

ICMP – Comando Traceroute

- Consente di conoscere il percorso che un pacchetto IP effettua per raggiungere un'host destinatario (man traceroute)
 - Per testarlo sulla macchina virtuale, cambiare le impostazioni di VirtualBox, per quanto riguarda la rete, scheda con bridge, invece che NAT
 - `sudo apt install traceroute`
 - `traceroute www.unipi.it`



ICMP – Traceroute

- Ogni pacchetto IP ha un campo TTL che i router decrementano di 1. Quando scende a 0 il router scarta il pacchetto e invia al mittente un messaggio ICMP TTL Exceeded.
- traceroute sfrutta questo meccanismo inviando alla destinazione pacchetti UDP (chiamati sonde) con il campo TTL crescente
- Ci possono essere diverse modalità per inviare le “sonde”
 - **UDP** - La sonda è un pacchetto UDP con numero di porta alto, che sarà quasi sicuramente chiusa. Gli hop intermedi rispondono con ICMP Time Exceeded. L'host finale, non avendo un processo in ascolto su quella porta, risponde con **ICMP Destination Unreachable, code Port Unreachable**. Quella risposta segnala a traceroute che hai raggiunto la destinazione.
 - ICMP Echo - La sonda è un ICMP Echo Request. Gli hop intermedi rispondono con Time Exceeded. La destinazione risponde con ICMP Echo Reply.
 - TCP SYN - Utile quando ICMP o UDP sono filtrati. La sonda è un SYN verso una porta specifica (80 o 443). Gli hop intermedi rispondono con Time Exceeded. La destinazione risponde con SYN/ACK oppure RST.

ICMP – Traceroute

- traceroute sfrutta questo meccanismo inviando alla destinazione **3** pacchetti UDP con il campo TTL crescente
 - Robustezza statistica, un drop occasionale può far saltare un hop. Tre tentativi riducono i falsi negativi.
 - Stima della variabilità di RTT.
 - Riconoscere load balancing per-packet. Se i tre IP a pari TTL cambiano, ci può bilanciamento sulla rotta.
 - Rate limiting sugli ICMP. Se un router risponde solo a una delle tre, capisci che limita il passaggio di ICMP.
- Il primo pacchetto con TTL=1, il secondo con TTL=2, ecc.
- Quindi ricostruisce il percorso usando i pacchetti Time Exceeded che riceve
 - A meno che qualche pacchetto non venga ricevuto (*)

ICMP – Traceroute

- Un asterisco indica che per quella sonda non è arrivata alcuna risposta entro il timeout
 - A volte le risposte arrivano con un tipo di "sonda" diverso
 - `traceroute -I www.google.com`
 - usa ICMP Echo
 - `traceroute -T -p 443 www.google.com`
 - usa TCP SYN

```
traceroute www.google.com
traceroute to www.google.com (216.58.205.196), 30 hops max, 60 byte packets
 1 131.114.175.73 (131.114.175.73)  0.348 ms  0.351 ms  0.378 ms
 2 jser-jing.unipi.it (131.114.191.129)  0.363 ms  0.371 ms  0.398 ms
 3 ru-unipi-rx1-pil.pil.garr.net (193.206.136.13)  1.302 ms  1.312 ms  1.321 ms
 4 rx1-pil-rx2-mi2.mi2.garr.net (90.147.80.210)  6.881 ms  6.899 ms  6.898 ms
 5 72.14.214.105 (72.14.214.105)  7.023 ms  6.974 ms  6.986 ms
 6 * * *
 7 216.239.42.27 (216.239.42.27)  7.020 ms  6.931 ms  6.997 ms
 8 mil04s29-in-f4.1e100.net (216.58.205.196)  6.949 ms  7.090 ms  7.087 ms
```

ANALISI DEI PACCHETTI

Analisi dei pacchetti

- Un'interfaccia collegata a una rete è in grado di ricevere e visualizzare tutti i pacchetti che circolano sul mezzo condiviso
- Di solito, l'interfaccia conserva e analizza solamente quelli destinati a lei.
 - Cioè quelli che hanno nel campo destinazione il suo indirizzo MAC
- Settando l'interfaccia in modalità promiscua, vengono analizzati tutti i pacchetti
- Comando **tcpdump** (man tcpdump)
 - È un software che setta l'interfaccia in modalità promiscua e visualizza tutti i pacchetti che circolano sulla rete locale, compresi quelli inviati e ricevuti dall'interfaccia
 - `sudo tcpdump [opzioni] [espressione]`

Analisi dei pacchetti - tcpdump

■ Opzioni

- **-c** (*count*)
 - Specifica il numero di pacchetti da visualizzare
- **-i** (*interface*)
 - Specifica l'interfaccia da usare
- **-q** (*quick/quiet*)
 - Visualizza meno informazioni
- **-n**
 - non risolve i nomi DNS (più veloce e chiaro)
- **-v, -vv, -vvv**
 - livello di dettaglio crescente
- **-w nome_file.pcap** (*write*)
 - Scrive l'output in un file (da aprire poi con Wireshark)
- **-r nome_file.pcap** (*read*)
 - Legge un file precedentemente creato con **-w**

Analisi dei pacchetti - tcpdump

- L'espressione serve per filtrare i pacchetti (man pcap-filter)
 - `sudo tcpdump port 80`
 - `sudo tcpdump host 192.168.4.2`
 - `sudo tcpdump udp and port 5555`
 - `sudo tcpdump dst host google.it and port 80`
 - `sudo tcpdump src host 192.168.0.2 and not port 90`
- Per filtrare pacchetti di richiesta DHCP
 - `sudo tcpdump -i enp0s3 port 67 or port 68 -n -vv`